

Recursion & Tree Recursion

Building the Mental Model — Seeing the Pattern, Not Memorizing the Code

A visual, intuition-first guide in Python

From single-branch recursion to backtracking & multi-way trees

Every section: teaching → practice problems → solutions

Contents

1. The Mental Model: What Recursion Really Is
 - a. The two questions
 - b. The call stack, drawn
 - c. Trust the recursion (the leap of faith)
2. Linear Recursion: One Branch, One Path
 - a. Factorial, sum, reverse, power
 - b. The universal template
 - c. Practice & solutions
3. Tree Recursion: When One Call Becomes Two
 - a. Fibonacci and the exploding tree
 - b. Why it is called a *tree*
 - c. Overlapping work & memoization
 - d. Practice & solutions
4. Recursion on Actual Trees
 - a. The node mental model
 - b. Sum, depth, leaves, traversal
 - c. The "return up" pattern
 - d. Practice & solutions
5. Branching Recursion: Subsets & Permutations
 - a. Choose / don't choose
 - b. The permutation tree
 - c. Practice & solutions
6. Backtracking: Recursion That Undoes Itself
 - a. The path + prune pattern
 - b. Combination sum, parentheses, N-Queens
 - c. Practice & solutions
7. Pattern Recognition Cheat Sheet

1 The Mental Model: What Recursion Really Is

Before any code, install this one idea: **a recursive function solves a big problem by asking a slightly smaller version of itself for help, and trusting that it answers correctly.** You never trace the whole thing in your head. You describe one honest step and let the machine repeat it.

Think about a line of people who want to know their position in a queue. You can't see the front. So you tap the person ahead: "What's your number?" They don't know either, so they tap the person ahead of *them*. This continues until it reaches the very first person, who *knows* they are number 1 without asking anyone. That answer then flows back: $1 \rightarrow 2 \rightarrow 3 \rightarrow \dots \rightarrow \text{you}$. That is recursion. The person who knows the answer without asking is the **base case**. Everyone else does the **recursive step**: `my_number = person_ahead() + 1`.

1a The two questions

Every recursive function you will ever write answers exactly two questions. Train yourself to ask them in this order, every single time:

THE ONLY TWO QUESTIONS

- 1. Base case:** What is the smallest input where I know the answer immediately, with no further help? (empty list, `n == 0`, a null node, a single character.)
- 2. Recursive step:** Assuming a smaller version already gave me the correct answer, how do I combine it with the current piece to build *my* answer?

Notice question 2 does not say "figure out the smaller version." It says *assume* it is already solved. This is the mental shift that makes recursion click. You are not thinking ten levels deep. You are thinking exactly one level deep and trusting the rest.

1b The call stack, drawn

The computer keeps a stack of paused function calls. Each call waits, holding its own local values, until the call it is waiting on returns. Here is `factorial(3)` as it grows *down* (calls being made) and then collapses back *up* (answers returning):

GOING DOWN (each call pauses, asks a smaller one)	COMING BACK UP (answers combine)
<code>factorial(3)</code> waits for <code>factorial(2)</code>	<code>factorial(3) = 3 * 2 = 6</code> <-- final ^ returns 6
<code>factorial(2)</code> waits for <code>factorial(1)</code>	 <code>factorial(2) = 2 * 1 = 2</code> ^ returns 2
<code>factorial(1)</code> waits for <code>factorial(0)</code>	 <code>factorial(1) = 1 * 1 = 1</code> ^ returns 1
<code>factorial(0) = 1</code> BASE CASE returns 1 ----->	<code>factorial(0) = 1</code>

Read it as two motions. On the way **down**, nothing is computed yet — every call is stuck waiting, stacking up. The instant we hit the base case `factorial(0) = 1`, the direction flips. On the way **up**, each waiting call receives its answer, multiplies in its own `n`, and returns. The "computation" of a recursive function largely happens on the way back up.

1c Trust the recursion (the leap of faith)

The single biggest beginner mistake is trying to simulate every level mentally. Don't. When you write the recursive step, adopt this belief:

THE LEAP OF FAITH

"I will assume the recursive call on the *smaller* input returns the correct answer. My only job is to (a) make the input genuinely smaller so it heads toward the base case, and (b) combine that returned answer with my current piece."

If both are true — smaller input each time, and a correct combine — the whole thing is correct by induction. You proved the base case works, and you proved that *if* the smaller case works *then* yours works. That covers all sizes. This is why recursion and mathematical induction are the same idea wearing different clothes.

THE #1 BUG

No base case, or an input that doesn't shrink toward it → infinite recursion → `RecursionError: maximum recursion depth exceeded`. Before running, always check: "Does every recursive call move strictly closer to a base case?"

2 Linear Recursion: One Branch, One Path

Start where recursion is simplest: each call makes *at most one* recursive call. The stack is a straight rope, not a tree. Master this shape first — tree recursion is just this idea with the rope splitting.

2a Four canonical examples

Factorial. The base case is $0! = 1$. The step: $n!$ is just n times the smaller factorial $(n-1)!$.

```
def factorial(n):
    if n == 0:                # base case: known instantly
        return 1
    return n * factorial(n - 1) # step: combine n with the smaller answer

factorial(5)    # -> 120
```

Sum of a list. A powerful reframe: a list is just *its first element plus the rest of the list*. The rest is a smaller list — that's your recursive input.

```
def sum_list(nums):
    if not nums:              # base case: empty list sums to 0
        return 0
    return nums[0] + sum_list(nums[1:]) # first + sum(rest)

sum_list([3, 1, 4, 1, 5])    # -> 14
```

```
sum_list([3,1,4,1,5])
= 3 + sum_list([1,4,1,5])
  = 1 + sum_list([4,1,5])
    = 4 + sum_list([1,5])
      = 1 + sum_list([5])
        = 5 + sum_list([])  <-- base case returns 0
          = 5 + 0 = 5
        = 1 + 5 = 6
      = 4 + 6 = 10
    = 1 + 10 = 11
  = 3 + 11 = 14
```

Reverse a string (real use: it's the shape behind "process last-in-first-out"). Move the first character to the end, reverse the rest.

```
def reverse(s):
    if len(s) <= 1:
        return s
    return reverse(s[1:]) + s[0] # reverse(rest) then stick first char on the end

reverse("hello")    # -> "olleh"
```

Power. base^{exp} is base times $\text{base}^{(\text{exp}-1)}$.

```
def power(base, exp):
    if exp == 0:
        return 1
    return base * power(base, exp - 1)

power(2, 10)    # -> 1024
```

2b The universal template

Every linear recursion above is the same skeleton. Internalize this shape and 80% of linear problems become fill-in-the-blank:

```
def solve(problem):
    if is_smallest(problem):           # 1. BASE CASE
        return known_answer
    smaller = make_smaller(problem)    # 2. SHRINK (must move toward base)
    return combine(current_piece, solve(smaller)) # 3. COMBINE
```

PATTERN SIGNAL — REACH FOR LINEAR RECURSION WHEN...

...the problem is naturally "one item + a smaller version of the same problem." Lists (head + tail), numbers (a digit + the rest), or any structure you can peel one layer off. If peeling gives you **one** smaller subproblem, it's linear.

■ Practice — Section 2 (solutions on the next page)

1. **Sum of digits.** Write `sum_digits(n)` so `sum_digits(12345)` returns `15`. Peel one digit at a time. (Hint: `n % 10` is the last digit, `n // 10` is the rest.)
2. **Count a character.** Write `count_char(s, c)` returning how many times `c` appears in `s`.
`count_char("banana", "a")` → `3`.
3. **GCD (Euclid).** Write `gcd(a, b)` using `gcd(a, b) = gcd(b, a % b)` with base case `b == 0`.
`gcd(48, 36)` → `12`. (Real use: reducing fractions.)
4. **Palindrome check.** Write `is_pal(s)` that peels from *both* ends. `is_pal("racecar")` → `True`,
`is_pal("hello")` → `False`.

Solutions — Section 2

1. Sum of digits

```
def sum_digits(n):
    if n < 10:                # base: single digit is its own sum
        return n
    return n % 10 + sum_digits(n // 10) # last digit + sum(rest)

sum_digits(12345)    # -> 15
```

Why it works: `12345 % 10 = 5`, then recurse on `1234`. Each call strips one digit, so it always shrinks toward the single-digit base case.

2. Count a character

```
def count_char(s, c):
    if not s:
        return 0
    return (1 if s[0] == c else 0) + count_char(s[1:], c)

count_char("banana", "a")    # -> 3
```

The head contributes 0 or 1; the tail contributes the rest. Same "head + smaller" shape as `sum_list`.

3. GCD (Euclid's algorithm)

```
def gcd(a, b):
    if b == 0:
        return a
    return gcd(b, a % b)

gcd(48, 36)    # -> 12
```

```
gcd(48, 36) -> gcd(36, 12) -> gcd(12, 0) -> 12
              (48%36=12)   (36%12=0)      base case
```

The second argument strictly decreases every call (a modulo is always smaller than the divisor), guaranteeing we reach `b == 0`.

4. Palindrome check (two-ended peel)

```
def is_pal(s):  
    if len(s) <= 1:           # empty or single char: trivially a palindrome  
        return True  
    if s[0] != s[-1]:         # ends differ -> not a palindrome  
        return False  
    return is_pal(s[1:-1])    # ends match -> check the inside  
  
is_pal("racecar")    # -> True  
is_pal("hello")      # -> False
```

Here "smaller" means chopping two characters (both ends). Still one recursive call, so still linear — the shrink step just removes more than one element.

3 Tree Recursion: When One Call Becomes Two

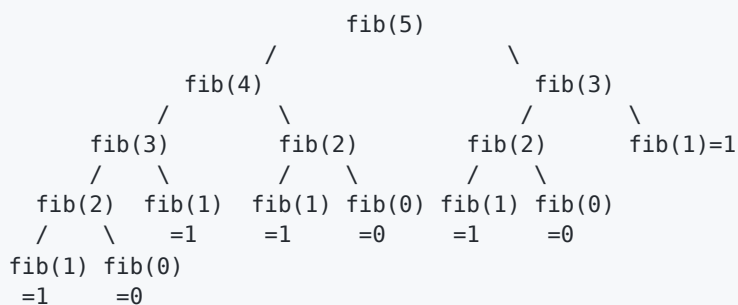
Now the key move. In linear recursion each call made one call — a straight line. **Tree recursion is when a single call makes two (or more) recursive calls.** The stack stops being a rope and becomes a branching tree. This is the shape that powers Fibonacci, subsets, permutations, backtracking, and traversing real trees — everything advanced.

3a Fibonacci and the exploding tree

Each Fibonacci number is the sum of the two before it: $F(n) = F(n-1) + F(n-2)$. That single line contains two recursive calls. Watch what happens:

```
def fib(n):  
    if n < 2:                # base cases: fib(0)=0, fib(1)=1  
        return n  
    return fib(n - 1) + fib(n - 2) # TWO calls -> the tree branches  
  
fib(10)    # -> 55
```

Draw `fib(5)`. Every node splits into two children until it reaches a base case (a leaf, `0` or `1`):



Read UP from the leaves: $\text{fib}(2)=1+0=1$, $\text{fib}(3)=1+1=2$, $\text{fib}(4)=2+1=3$, $\text{fib}(5)=3+2=5$

3b Why it is called a tree

Literally look at the diagram — it is a tree, drawn by the recursion itself. Each function call is a node; its recursive calls are its children; base cases are leaves. This is the whole reason the term is "tree recursion." Once you *see* the branching, you can reason about it: the number of leaves, the depth, the total work.

THE HIDDEN COST: OVERLAPPING SUBPROBLEMS

Look again at the `fib(5)` tree. `fib(3)` is computed twice. `fib(2)` is computed three times. The tree recomputes the same values over and over. `fib(10)` makes 177 calls; `fib(50)` would make billions. Naive tree recursion is exponential: about $O(2^n)$.

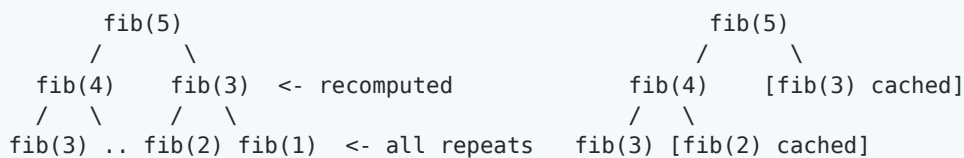
3c Memoization — cache the branches you've already grown

The fix is simple and it is one of the most important ideas in all of algorithms: **remember answers you've already computed.** Store each result in a dictionary; before recursing, check if you already know it.

```
def fib_memo(n, memo=None):
    if memo is None:
        memo = {}
    if n < 2:
        return n
    if n in memo:
        # already solved? return instantly
        return memo[n]
    memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
    return memo[n]

fib_memo(50) # -> 12586269025 (instant)
```

Without memo, fib(5) tree has many repeats: With memo, each fib(k) computed ONCE:



Calls drop from $O(2^n)$ exponential $\rightarrow O(n)$ linear. Same code shape, a cache bolted on.

PATTERN SIGNAL — REACH FOR TREE RECURSION WHEN...

...solving the problem requires combining answers from **two or more** smaller subproblems (left and right, take and skip, this branch and that branch). The moment your recursive step has two recursive calls added/joined together, you are in a tree. And if those subproblems *overlap*, add memoization.

■ Practice — Section 3 (solutions on the next page)

1. **Count the calls.** Without running it, is naive `fib(n)` closer to $O(n)$ or $O(2^n)$? Explain using the tree. Then state what memoization changes it to.
2. **Climbing stairs.** You climb `n` stairs taking 1 or 2 steps at a time. The number of distinct ways is `ways(n) = ways(n-1) + ways(n-2)` (same shape as Fibonacci!). Write `climb(n)` with memoization. `climb(5) → 8`. (Real use: this "how many ways to reach a state" pattern is everywhere in dynamic programming.)
3. **Draw the tree.** On paper, draw the call tree for `climb(4)`. Mark which nodes get recomputed without a cache.

Solutions — Section 3

1. Counting the calls

Naive `fib(n)` is $O(2^n)$ (exponential). Reason from the tree: each non-base node spawns two children, so the number of nodes roughly doubles with each extra level of depth. Concretely, `fib(10)` makes 177 calls, and the count nearly doubles for each `+1` to `n`. With memoization each distinct `fib(k)` for `k = 0..n` is computed exactly once, so it becomes $O(n)$ — linear.

2. Climbing stairs (memoized)

```
def climb(n, memo=None):
    if memo is None:
        memo = {}
    if n <= 2:                # 1 stair:1 way, 2 stairs:2 ways
        return n
    if n in memo:
        return memo[n]
    memo[n] = climb(n - 1, memo) + climb(n - 2, memo)
    return memo[n]

climb(5)    # -> 8
```

Intuition for the recurrence: to reach stair `n`, your last move was either a 1-step (from stair `n-1`) or a 2-step (from stair `n-2`). So the ways to reach `n` = ways to reach `n-1` + ways to reach `n-2`. Recognizing that "last move" split is the whole trick.

3. The climb(4) tree

```
      climb(4)
     /      \
  climb(3)  climb(2)=2  <- base
   /  \
climb(2)=2 climb(1)=1    <- both base
```

`climb(2)` appears on the left branch AND as `climb(4)`'s right child. Without a cache, `climb(2)` is recomputed. WITH memo, the second time it is a dictionary lookup. Result: `climb(4) = 3 + 2 = 5`.

The repeated `climb(2)` is exactly the overlapping-subproblem waste that memoization eliminates.

4 Recursion on Actual Trees

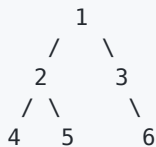
Sections 2 and 3 used recursion whose *call structure* looked like a tree. Now the data itself is a tree. This is where tree recursion feels most natural — the shape of your code mirrors the shape of your data. Real uses are everywhere: a file system (folders inside folders), an HTML/DOM page, a company org chart, a decision tree in an ML model, JSON objects.

4a The node mental model

A binary tree node holds a value and links to (up to) two children. Being `None` means "no node here" — that is almost always your base case.

```
class TreeNode:
    def __init__(self, val, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# The tree we'll use throughout this section:
root = TreeNode(1,
    TreeNode(2, TreeNode(4), TreeNode(5)),
    TreeNode(3, None, TreeNode(6)))
```



Node 1 has children 2 and 3. Node 3 has no left child, right child 6.
Nodes 4, 5, 6 are leaves (no children).

THE MASTER TEMPLATE FOR TREE PROBLEMS

Nearly every binary-tree recursion is this: **if the node is `None`, return the empty/zero answer; otherwise combine `node.val` with the recursive answers from `node.left` and `node.right`.** Two recursive calls per node — that is why tree traversal is tree recursion.

4b The four workhorses

Sum every value. A tree's sum is its root value plus the sum of the left subtree plus the sum of the right subtree.

```
def tree_sum(node):
    if node is None:                # base: empty tree sums to 0
        return 0
    return node.val + tree_sum(node.left) + tree_sum(node.right)

tree_sum(root)    # -> 21    (1+2+3+4+5+6)
```

```

tree_sum(1)
= 1 + tree_sum(2) + tree_sum(3)
    |           |
    = 2 + sum(4) + sum(5)    = 3 + sum(None) + sum(6)
      = 2 + 4   + 5   = 11    = 3 + 0   + 6   = 9
= 1 + 11 + 9 = 21

```

Maximum depth. The depth of a tree is 1 (for the current node) plus the deeper of its two subtrees.

```

def max_depth(node):
    if node is None:
        return 0
    return 1 + max(max_depth(node.left), max_depth(node.right))

max_depth(root)    # -> 3

```

Count leaves. A leaf is a node with no children. This has an extra base case.

```

def count_leaves(node):
    if node is None:
        return 0
    if node.left is None and node.right is None:
        return 1 # this node is a leaf
    return count_leaves(node.left) + count_leaves(node.right)

count_leaves(root)    # -> 3    (leaves: 4, 5, 6)

```

In-order traversal. Visit left subtree, then the node, then right subtree — concatenate the lists.

```

def inorder(node):
    if node is None:
        return []
    return inorder(node.left) + [node.val] + inorder(node.right)

inorder(root)    # -> [4, 2, 5, 1, 3, 6]

```

4c The "return up" pattern

The unifying idea: each node asks its two children for their answers, then *merges* and returns its own answer upward. Information flows from the leaves back to the root. Once you see this "ask both children, combine, return up" rhythm, every tree problem is a variation of it — you only change the *combine* step (add for sum, max for depth, concatenate for traversal).

PATTERN SIGNAL — YOU HAVE A TREE PROBLEM WHEN...

...the data is a hierarchy (children inside parents) OR the question is "for this node, what's the answer given the answers of its subtrees?" The base case is almost always `None` → identity value (0 for sums, [] for lists, 0 for depth, True/False for logical checks).

■ **Practice — Section 4 (solutions on the next page)**

1. **Find the max value** in the tree. Write `tree_max(node)`. For `root` above it returns `6`. (Hint: base case for `None` should be negative infinity, `float('-inf')`, so it never wins the max.)
2. **Count all nodes** (not just leaves). `count_nodes(root)` → `6`.
3. **Root-to-leaf path sum**. Write `has_path_sum(node, target)` returning `True` if some root-to-leaf path's values add up to `target`. For our tree, path `1 → 2 → 4` sums to 7, so `has_path_sum(root, 7)` → `True`; `has_path_sum(root, 100)` → `False`. (Real use: this is exactly how you search decision trees.)

Solutions — Section 4

1. Max value in the tree

```
def tree_max(node):
    if node is None:
        return float('-inf')    # empty branch can never be the max
    return max(node.val, tree_max(node.left), tree_max(node.right))

tree_max(root)    # -> 6
```

Same "ask both children, combine" template — the combine step is just `max` instead of `+`. Using `-inf` as the empty answer is the identity element for max, exactly like `0` is for sum.

2. Count all nodes

```
def count_nodes(node):
    if node is None:
        return 0
    return 1 + count_nodes(node.left) + count_nodes(node.right)

count_nodes(root)    # -> 6
```

Count myself (1) plus everything in my two subtrees. This is `tree_sum` with every value replaced by 1.

3. Root-to-leaf path sum

```
def has_path_sum(node, target):
    if node is None:
        return False
    if node.left is None and node.right is None:    # at a leaf
        return node.val == target
    remaining = target - node.val                    # subtract as we go down
    return (has_path_sum(node.left, remaining) or
            has_path_sum(node.right, remaining))

has_path_sum(root, 7)    # -> True    (1 -> 2 -> 4)
has_path_sum(root, 100)  # -> False
```

```
has_path_sum(1, 7)  -> need 7-1=6 below
has_path_sum(2, 6) -> need 6-2=4 below
has_path_sum(4, 4) -> leaf, 4==4  TRUE  <-- found it, short-circuits via `or`
```

Two new ideas here: (1) we thread state downward by subtracting `node.val` from the target as we descend, and (2) `or` means "a path exists if EITHER subtree contains one" — and it short-circuits, stopping as soon as one succeeds.

5 Branching Recursion: Subsets & Permutations

Here tree recursion becomes a tool for *generating all possibilities*. The recursion tree isn't incidental — it *is* the answer. Each root-to-leaf path is one complete possibility. Real uses: generating test cases, exploring configurations, feature selection, planning move sequences in games.

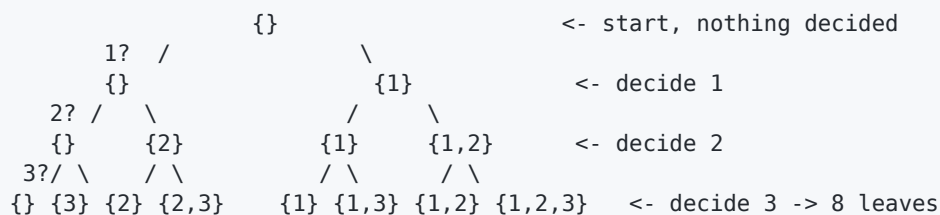
5a Subsets — the "choose / don't choose" tree

Every element faces a binary decision: is it in the subset, or not? That's two branches per element — a binary tree of decisions. One clean way to see it: the subsets of `[first] + rest` are exactly (all subsets of `rest`) plus (each of those with `first` added in).

```
def subsets(nums):
    if not nums:                # base: the only subset of [] is the empty set
        return [[]]
    first = nums[0]
    rest = subsets(nums[1:])    # all subsets WITHOUT first
    return rest + [[first] + s for s in rest] # plus those same subsets WITH first

subsets([1, 2, 3])
# -> [[], [3], [2], [2,3], [1], [1,3], [1,2], [1,2,3]] (8 = 2^3 subsets)
```

Decision tree for [1,2,3] (L = skip element, R = include element):



Every one of the 8 leaves is a complete subset. n elements -> 2^n leaves.

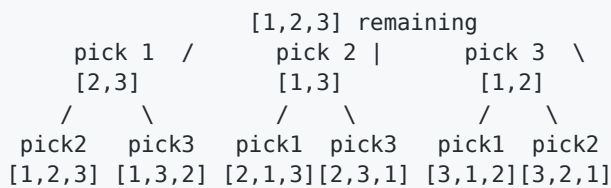
5b Permutations — pick one, permute the rest

Permutations are *orderings*. The tree branches differently: at each step, pick *any remaining* element to go next, then permute what's left. So a node with `k` remaining elements has `k` children (not 2).

```
def permutations(nums):
    if len(nums) <= 1:
        return [nums]
    result = []
    for i in range(len(nums)):
        rest = nums[:i] + nums[i+1:] # everything except the chosen element
        for p in permutations(rest): # permute the remainder (trust it!)
            result.append([nums[i]] + p) # put chosen element in front
    return result

permutations([1, 2, 3])
# -> [[1,2,3], [1,3,2], [2,1,3], [2,3,1], [3,1,2], [3,2,1]] (6 = 3! orderings)
```


Permutation tree for [1,2,3] (branch = "which element goes next"):



3 choices, then 2, then 1 -> $3! = 6$ leaves. This is why it's $O(n!)$.

PATTERN SIGNAL — "GENERATE ALL ____"

Any "list every possible ____" problem is branching recursion. Ask: **at each step, what are my choices, and what smaller problem remains after each choice?** Subsets → 2 choices per element (in/out). Permutations → k choices (which comes next). Combinations, board configurations, etc. all fit this frame.

■ Practice — Section 5 (solutions on the next page)

1. **Phone letter combinations.** On a phone keypad, $2 \rightarrow abc$, $3 \rightarrow def$, etc. Given a digit string, generate all letter strings. `letter_combos("23")` should produce 9 strings (3×3). This is a branching recursion where each digit's branch count equals its number of letters.
2. **Generate binary strings.** Write `binary_strings(n)` that returns all strings of `0` / `1` of length `n`. `binary_strings(2) → ['00', '01', '10', '11']`. (This is the pure "choose/don't-choose" tree in its simplest form.)
3. **Count without generating.** Without building the lists, how many subsets does a 10-element set have? How many permutations? State the formula and the number.

Solutions — Section 5

1. Phone letter combinations

```
def letter_combos(digits):
    if not digits:
        return []
    mapping = {'2':'abc', '3':'def', '4':'ghi', '5':'jkl',
               '6':'mno', '7':'pqrs', '8':'tuv', '9':'wxyz'}
    result = []
    def backtrack(idx, path):
        if idx == len(digits):          # used every digit -> complete combo
            result.append(path)
            return
        for ch in mapping[digits[idx]]: # branch once per possible letter
            backtrack(idx + 1, path + ch)
    backtrack(0, "")
    return result

len(letter_combos("23"))  # -> 9  ('ad','ae','af','bd',...,'cf')
```

Each level of the tree corresponds to one digit; the branching factor at that level is how many letters the digit maps to. This `backtrack(idx, path)` shape is the bridge into Section 6.

2. Generate binary strings

```
def binary_strings(n):
    if n == 0:
        return [""]          # one empty string
    smaller = binary_strings(n - 1)  # trust: all strings of length n-1
    return ["0" + s for s in smaller] + ["1" + s for s in smaller]

binary_strings(2)  # -> ['00','01','10','11']
```

Prepend "0" to every shorter string, then prepend "1" to every shorter string. That's the choose/don't-choose split in its cleanest form — and it produces exactly 2^n results.

3. Counting without generating

A 10-element set has $2^{10} = 1024$ subsets: each element is independently in or out, so multiply 2 by itself 10 times. It has $10! = 3,628,800$ permutations: 10 choices for first position, then 9, then 8, ... down to 1, giving the factorial. This is why "generate all subsets" is fine for small n but "generate all permutations" blows up fast — factorial growth outpaces even exponential.

6 Backtracking: Recursion That Undoes Itself

Backtracking is the most powerful form of tree recursion, and it's what most hard interview and real-world problems reduce to. The idea: explore a choice, recurse deeper, and if it leads nowhere useful, **undo the choice** and try the next one. You are walking a decision tree, but you carry one shared "path" and mutate it as you go — pushing when you descend, popping when you back out.

6a The path + prune pattern

Three moving parts define every backtracking solution. Memorize this skeleton — it is the single most reusable template in this entire guide:

```
def solve(...):
    result = []
    def backtrack(state, path):
        if is_complete(state):          # 1. GOAL: a full solution -> record it
            result.append(list(path))
            return
        for choice in choices(state):   # 2. try each option
            if not valid(choice): continue # PRUNE dead ends early
            path.append(choice)          # CHOOSE (descend)
            backtrack(advance(state), path)
            path.pop()                  # UN-CHOOSE (the "backtrack" step!)
    backtrack(start, [])
    return result
```

THE HEARTBEAT OF BACKTRACKING

choose → **explore** → **un-choose**. The `path.append(...)` before the recursive call and the matching `path.pop()` after it are the essence. The pop is what lets one shared `path` list explore the entire tree without leftover state from abandoned branches.

6b Combination sum — pick numbers that add to a target

Given candidate numbers, find all combinations that sum to a target (numbers reusable). Real use: coin/change problems, resource allocation.

```
def combination_sum(candidates, target):
    result = []
    def backtrack(start, path, remaining):
        if remaining == 0: # exact hit -> record this combination
            result.append(list(path))
            return
        if remaining < 0: # overshoot -> PRUNE, dead branch
            return
        for i in range(start, len(candidates)): # start avoids duplicate combos
            path.append(candidates[i])
            backtrack(i, path, remaining - candidates[i]) # i (not i+1): reuse allowed
            path.pop() # UNDO
    backtrack(0, [], target)
    return result

combination_sum([2, 3, 6, 7], 7) # -> [[2,2,3],[7]]
```

backtrack exploring target=7 with [2,3,6,7]:

```
path=[]    rem=7
+2 -> [2]   rem=5
    +2 -> [2,2] rem=3
        +3 -> [2,2,3] rem=0 RECORD [2,2,3]
    +3 -> [2,3] rem=2 -> nothing hits 0 -> pop back
+7 -> [7]   rem=0 RECORD [7]
```

Each 'pop' rewinds the path so the next branch starts clean.

6c Valid parentheses — pruning with constraints

Generate all valid combinations of `n` pairs of parentheses. The pruning is the clever part: only add `)` if it can still be matched.

```
def gen_paren(n):
    result = []
    def backtrack(s, open_c, close_c):
        if len(s) == 2 * n: # used all n pairs -> complete
            result.append(s)
            return
        if open_c < n: # can still open
            backtrack(s + "(", open_c + 1, close_c)
        if close_c < open_c: # only close what's already open (PRUNE)
            backtrack(s + ")", open_c, close_c + 1)
    backtrack("", 0, 0)
    return result

gen_paren(3) # -> ['((()))', '(()())', '(())()', '()(())', '()()()'] (5 valid)
```

Here the "path" is the string `s`, and instead of append/pop we build new strings — but it's the same tree. The two `if` guards are pruning: they refuse to even explore branches that can't lead to valid output. Good pruning is what turns an intractable tree into a fast one.

6d N-Queens — the classic

Place `n` queens on an `n×n` board so none attack each other. Place one queen per row; for each row try every column, skip columns/diagonals already threatened, recurse, then undo.

```
def total_n_queens(n):
    count = 0
    cols, diag1, diag2 = set(), set(), set()  # threatened columns and two diagonals
    def backtrack(row):
        nonlocal count
        if row == n:  # placed a queen in every row -> a solution
            count += 1
            return
        for col in range(n):
            if col in cols or (row - col) in diag1 or (row + col) in diag2:
                continue  # PRUNE: this square is attacked
            cols.add(col); diag1.add(row - col); diag2.add(row + col)  # CHOOSE
            backtrack(row + 1)
            cols.remove(col); diag1.remove(row - col); diag2.remove(row + col)  # UNDO
        backtrack(0)
    return count

total_n_queens(4)  # -> 2
total_n_queens(8)  # -> 92
```

Cells on the same "↗" diagonal share `row + col`; cells on the same "↘" diagonal share `row - col`. Storing those in sets makes the attack check $O(1)$. Note the perfect choose/undo symmetry — every `.add` before recursing has a matching `.remove` after.

PATTERN SIGNAL — REACH FOR BACKTRACKING WHEN...

...you must **build a solution incrementally out of choices**, and (a) you want *all* valid solutions, or (b) you're searching for *any* solution under constraints. Tell-tale words: "all combinations/arrangements/ways," "place/assign subject to rules," "partition," "find a valid configuration." If you can phrase it as "make a choice, recurse, undo," it's backtracking.

■ Practice — Section 6 (solutions on the next page)

1. **Subsets, backtracking style.** Re-implement subset generation using the choose/un-choose template (append then pop) instead of the Section-5 slicing approach. Confirm it yields 8 subsets for `[1,2,3]`.
2. **Permutations, backtracking style.** Generate permutations of `[1,2,3]` using a `used` set to track which elements are already in the path. Should yield 6.
3. **Explain the pop.** In one or two sentences, explain what breaks if you forget the `path.pop()` (or the `.remove()` in N-Queens). Why is the undo mandatory?

Solutions — Section 6

1. Subsets via backtracking

```
def subsets(nums):
    result = []
    def backtrack(start, path):
        result.append(list(path))      # every node is a valid subset, record it
        for i in range(start, len(nums)):
            path.append(nums[i])       # CHOOSE nums[i]
            backtrack(i + 1, path)     # explore further (no reuse)
            path.pop()                 # UN-CHOOSE
        backtrack(0, [])
    return result

len(subsets([1, 2, 3])) # -> 8
```

Unlike combination sum, we record at every node (each partial path is itself a valid subset), and use `i+1` so each element is used at most once.

2. Permutations via backtracking

```
def permutations(nums):
    result = []
    used = set()
    def backtrack(path):
        if len(path) == len(nums):    # path uses every element -> a full ordering
            result.append(list(path))
            return
        for x in nums:
            if x in used: continue     # skip elements already placed (PRUNE)
            used.add(x); path.append(x) # CHOOSE
            backtrack(path)
            path.pop(); used.remove(x)  # UNDO both
        backtrack([])
    return result

len(permutations([1, 2, 3])) # -> 6
```

The `used` set is what makes it a permutation rather than a subset: it forbids repeating an element, and every complete path of full length is one ordering.

3. Why the undo is mandatory

If you skip `path.pop()`, the choices from one branch leak into the next. After exploring "include 1," the `1` would still sit in `path` when you move to the "start with 2" branch, corrupting every sibling and producing wrong, overlong results. The pop rewinds shared state back to exactly what it was before the choice — restoring a clean slate so each sibling branch explores independently. In short: **append/pop must always come in matched pairs around the recursive call.**

7 Pattern Recognition Cheat Sheet

The goal of this whole guide: look at a new problem and instantly recognize its recursive shape. Use this table as the diagnostic. Read the left column, and the right columns tell you the shape and the template to reach for.

When you see...	Shape	Base case	Recursive step
"One item + a smaller version of the same problem" (lists, numbers, one-ended peel)	Linear (\$2)	empty / 0 / single element	one call: <code>combine(head, solve(tail))</code>
"Answer depends on two smaller answers" (F(n-1)+F(n-2), left+right)	Tree (\$3)	smallest known values	two calls, joined: <code>solve(a) + solve(b)</code>
Same subproblem solved many times (overlaps)	Tree + memo (\$3c)	same as tree	cache result in a dict before returning
Data is a hierarchy (tree/DOM/filesystem/JSON); "for this node given its subtrees"	Tree traversal (\$4)	<code>None</code> → identity (0, [], -inf, True)	<code>combine(node.val, rec(left), rec(right))</code>
"Generate ALL subsets / combinations / ways"	Branching (\$5)	empty input → <code>[]</code>	2 branches per element (in / out)
"Generate ALL orderings / arrangements"	Branching (\$5)	≤1 element → itself	k branches: pick each remaining element next
"Build a valid solution under constraints"; "all ways to place/assign/partition"	Backtracking (\$6)	path complete → record	choose → recurse → un-choose; prune invalid

The five-step attack for any recursive problem

DO THIS EVERY TIME

1. Ask: what is the *smaller version* of this exact problem? (Identify the shrink.)
2. Ask: what's the smallest input I can answer with zero effort? (The base case.)
3. Take the leap of faith: *assume* the recursive call on the smaller input is already correct.
4. Ask: how do I combine that assumed-correct answer(s) with my current piece? (One call → linear; two+ → tree; choices with undo → backtracking.)
5. Check termination: does every call move strictly toward the base case? If subproblems repeat, add memoization.

Three habits that build the mental model

1. **Always draw the tree first.** Before coding tree/branching/backtracking problems, sketch two or three levels of the recursion tree by hand. The code then writes itself from the picture.
2. **Trust, don't trace.** Resist simulating deep levels in your head. Verify the base case and the single combine step; induction guarantees the rest. Tracing beyond one level is how people confuse themselves.
3. **Name the identity value.** The base-case return is almost always the neutral element of your combine operation: `0` for `+`, `1` for `*`, `[]` for concatenation, `-inf` for `max`, `True` for `and`. Getting this right fixes most base-case bugs.

Every code example and solution in this document was executed and verified to produce the stated output. Work through the practice problems *before* reading the solutions — the mental model forms in the struggle, not in the reading.